Introdução à Linguagem C

Professor: André Felipe Ribeiro Cordeiro
Disciplina: Estruturas de Dados

Departamento de Informática
Centro de Tecnologia
Universidade Estadual de Maringá

Introdução

- A linguagem C foi criada por Dennis Ritchie, em 1972, no centro de Pesquisas da Bell Laboratories.

- É uma linguagem de propósito geral, sendo adequada à programação estruturada.

Sintaxe

- Todo programa em C inicia sua execução chamando a função **main()**, sendo obrigatória a sua declaração no programa principal.
- Comentários no código:

  /* comentário de mais

  de uma linha */.

  // comentário de uma linha
- Cada instrução encerra com **;** (ponto e vírgula), que faz parte do comando.

Sintaxe
Exemplo

```
#include <stdio.h>
// Função obrigatória em todos os programas escritos em C
void main() {
  // comando 1;
  // comando 2;
}
```

Observação: a função **main()** também pode retornar um **int**.

Tipos de Dados
Principais tipos primitivos

Tipo de dado	Significado	Tamanho (em bytes)	Intervalo de valores aceitos
char	Caractere (código ASCII)	1	de -128 a 127
short int	Inteiro curto	2	-de 32 768 a 32 767
unsigned short int	Inteiro curto sem sinal	2	de 0 a 65 535
int	Inteiro	4	de -2 147 483 648 a 2 147 483 647
unsigned int	Inteiro sem sinal	4	de 0 a 4 294 967 295
long int	Inteiro longo	4	-de 2 147 483 648 a 2 147 483 647
unsigned long int	Inteiro longo sem sinal	4	de 0 a 4 294 967 295
float	Flutuante (real)	4	$3.4*10^{-38}$ à $3.4*10^{38}$
double	Flutuante duplo	8	de $1.7*10^{-308}$ a $1.7*10^{308}$
long double	Flutuante duplo longo	10	$3.4*10^{-4932}$ à $3.4*10^{4932}$

Operadores Aritméticos e de Atribuição

Operador	Sintaxe
Adição	a + b
Subtração	a - b
Incremento	a++
Decremento	a--

Operador	Sintaxe
Divisão	a / b
Multiplicação	a * b
Módulo (resto)	a % b

Operador	Sintaxe
Atribuição	a = b

Operadores Comparativos

Operador	Sintaxe
Menor que	a < b
Menor ou igual que	a <= b
Maior que	a > b
Maior ou igual que	a >= b
Diferente de	a != b
Igual a	a == b
Não lógico	!a
E lógico	a && b
Ou lógico	a || b

Função printf()

printf("expressão de controle", argumentos);

- É uma função de *Input*/*Output*, que permite escrever na tela.
- A expressão de controle pode conter:
  - caracteres que serão exibidos na tela; e
  - códigos de formatação que indicam o formato em que os argumentos devem ser impressos.
- Cada argumento deve ser separado por vírgula.

Função printf()

Símbolo	Efeito
\n	Nova linha
\t	Tab
\b	Retrocesso
\”	Aspas

Símbolo	Significado
%c	Caractere simples
%d	Decimal
%f	Ponto flutuante
%s	Cadeia de caracteres

Função printf()

Exemplo

```
#include <stdio.h>
int main() {
 printf("Este e o numero dois: %d", 2);
 printf(" \n%s esta a %d milhoes de milhas
                        \ndo sol", "Venus", 67);
 return 0;
}
```

Função printf()

arredondamento

```
#include <stdio.h>
int main() {
 printf("\n%4.2f", 3456.78);
 printf("\n%4.1f", 3456.78);
 printf("\n%4.3f", 3456.78);
 return 0;
}
```

Função scanf()

- É o complemento de printf() e nos permite ler dados formatados da entrada padrão (teclado).

- Sua sintaxe é similar a printf().

```
scanf("expressão de controle", argumentos);
```

Função scanf()

- A lista de argumentos deve consistir nos **endereços das variáveis**.

- A linguagem C oferece um operador para tipos básicos chamado operador de endereço: **&**

Função scanf()

- A memória do computador é dividida em bytes, e são numerados de 0 até o limite da memória.
  - Estas posições são chamadas de endereços.
  - Toda variável ocupa uma certa localização na memória, e seu endereço é o primeiro byte ocupado por ela.

Função scanf()

```
#include <stdio.h>
void main() {
  int num;
  printf("Digite um numero: ");
  scanf("%d", &num);

  printf("\nNumero: %d", num);
  printf("\nEndereco: %d", &num);
}
```

Estrutura de Controle de Fluxo - IF

```
if (condição)
    comando;
else
    comando;

if (condição){
    comando;
} else {
    comando;
}
```

```
if   (condição)
{
    comando1;
    comando2;
    ...
} else {
    comando3;
    comando4;
    ...
}
```

```
if (condição) {
    comando1;
    ...
}      else      if
(condição) {
    comando2;
    ...
} else {
    ...
}
```

Estrutura de Controle de Fluxo - IF

```
#include <stdlib.h>
#include <time.h>
#include <stdio.h>
void main() {
  int num, segredo;
  srand(time(NULL));
  segredo = rand() / 10;
  printf("Qual e o numero: ");
  scanf("%d", &num);
  if (segredo == num) {
     printf("Acertou!");
     printf("\nO numero e %d\n", segredo);
  } else if (segredo < num) {
     printf("Errado, muito alto!\n");
  } else {
     printf("Errado, muito baixo!\n");
  }
}
```

Switch.. Case

```
switch (variável) {
    case constante1:
        sequência de comandos;
        break;
    case constante2:
        sequência de comandos;
        break;
    default:
        sequência de comandos;
}
```

Switch.. Case

```
#include <stdio.h>
void main() {
    char ch;
    printf("1. Inclusão\n");
    printf("2. Alteração\n");
    printf("3. Exclusão\n");
    printf(" Digite sua opção:");
    scanf("%c", &ch);
```

```
switch(ch) {
    case '1':
        printf("Escolheu inclusão.\n");
        break;
    case '2':
        printf("Escolheu alteração.\n");
        break;
    case '3':
        printf("Escolheu exclusão.\n");
        break;
    default:
        printf("Opção inválida.\n");
  }
}
```

Loop for

for (inicialização; condição; incremento)

```
#include <stdio.h>
void main() {
    int x;
    for (x = 1; x < 100; x++) {
        printf("%d\n", x);
    }
}
```

```
for (;;) {
    printf("loop infinito\n");
}
```

Loop for

```
#include <stdio.h>
void main() {
    int linha, coluna;
    for (linha = 1; linha <= 24; linha++){
        for (coluna = 1; coluna < 40; coluna++) {
            printf("-");
        }
        printf(“\n”);
    }
}
```

While

```
while  (condição)  {
comando;
}
```

```
do {
    comando;
} while(condição);
```

```
void main() {
    char ch;
    while (ch != 'a') {
        scanf("%c", &ch);
    }
}
```

Arrays

tipo nome[tamanho];

```
#include <stdio.h>
void main() {
    int x[10];
    int i;
    for(i = 0; i < 10; i++) {
        x[i] = i*2;
        printf("%d\n", x[i]);
    }
}
```

Arrays

```
#include <stdio.h>
void main() {
    int notas[5], i, soma;
    for(i = 0; i < 5; i++) {
        printf("Digite a nota do aluno %d: ", i);
        scanf("%d", &notas[i]);
    }
    soma = 0;
    for(i = 0; i < 5; i++)
        soma = soma+notas[i];
    printf("Media das notas: %d.", soma / 5);
}
```

Matrizes

tipo nome[tamanho][tamanho] ...;

```
#include <stdio.h>
void main() {
    int matriz[10][10];
    int i,j=0;
    for(i = 0; i < 10; i++, j++){
        matriz[i][j] = i*j;
        printf("%d\n", matriz[i][j]);
    }
}
```

Strings

- Em C não existe um tipo de dado string, no seu lugar é utilizado um **array de caracteres**.

- **Uma string é um array tipo char que termina com '\0'**.

- Por essa razão uma string deve conter uma posição a mais do que o número de caracteres que se deseja.

Função gets()

- Utilizada para leitura de uma string.

gets(nome_matriz);

```
#include <string.h>
#include <stdio.h>
void main() {
    char str[80];
    gets(str);
    printf("%s", str);
}
```

Função puts()

- Escreve o seu argumento na tela.

puts(nome_do_vetor_de_caracteres);

```
#include <string.h>

void main() {
    puts("mensagem");
}
```

Função strcpy()

- Copia o conteúdo de uma string.

strcpy(destino, origem);

```
#include <string.h>
void main(){
    char str[80];
    strcpy(str, “alo”);
    puts(str);
}
```

Função strcat()

- Concatena duas strings. Não verifica tamanho.

strcat(string1, string2);

```
#include <string.h>
void main(){
    char cad1[20], cad2[10];
    strcpy(cad1, “bom”);
    strcpy(cad2, “dia”);
    strcat(cad1, cad2);
    printf("%s\n", cad1);
}
```

Função strcmp()

- Compara duas strings, se forem iguais devolve 0.

strcmp(string1, string2);

```
#include <string.h>
void main(){
    char s[80];
    printf("Digite a senha:");
    gets(s);
    if (strcmp(s, "laranja") == 0)
       printf("senha ok\n");
    else
       printf("senha invalida!\n") ;
}
```

Observação: Em C não existe o tipo primitivo Booleano, desta forma, ele considera o valor inteiro 0 como falso e qualquer outro valor positivo como verdadeiro.

Ponteiros

- É uma variável que contém o endereço de outra variável.

- Os ponteiros são utilizados para alocação dinâmica de memória.

Ponteiros

- Também fornecem uma maneira pela qual as funções podem modificar os argumentos chamados.
  - Passagem de parâmetro por referência.

tipo *nomevar;

Ponteiros

- Se uma variável irá conter um ponteiro, então ela deve ser declarada como tal:

```
int x, *px;
px = &x; /* a variável px aponta para x */
```

- Se quisermos utilizar o conteúdo da variável para qual o ponteiro aponta:

```
y = *px;
```

- O que é a mesma coisa que:

```
y = x;
```

Ponteiros

```
void main(){
    int x, *px, *py;
    x = 9;
    px = &x;
    py = px;
    printf("x= %d\n", x);
    printf("&x= %d\n", &x);
    printf("px= %d\n", px);
    printf("*px= %d\n", *px);
}
```

Funções

```
int sqr(int x) {
    x = x*x;
    return x;
}

void main() {
    int t = 10;
    printf("%d %d", sqr(t), t);
}
```

Funções

```
void troca(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void main() {
    int x = 10, y = 20;
    troca(&x, &y);
    printf("x=%d y=%d\n", x, y);
}
```

Estruturas

- São agregados heterogêneos, compostos de vários tipos.

- As estruturas permitem uma organização dos dados dividida em campos e registros.

Estruturas

```
typedef struct {
    int dureza;
    char fabricante;
    int numero;
} lapis;

void main() {
    int i;
    struct lapis p[3];
    p[0].dureza = 2;
    p[0].fabricante = 'F';
    p[0].numero = 482;
    p[1].dureza = 0;
    p[1].fabricante = 'G';
    p[1].numero = 33;
    p[2].dureza = 3;
    p[2].fabricante = 'E';
    p[2].numero = 107;
    printf("Dureza
            Fabricante Numero\n");
    for(i = 0; i < 3; i++) {
       printf("%d\t%c\t\t%d\n",
           p[i].dureza,
       p[i].fabricante, p[i].numero);
    }
}
```

Noções de Alocação Dinâmica

```
#include <stdio.h>
void main() {
     int *p, t;
     p = (int*) malloc(40 * sizeof(int));
     if (!p) {
        printf("memoria insuficiente\n");
     } else {
        for (t = 0; t < 40; t++) {
           *(p + t) = t;
        }
        for (t = 0; t < 40; t++) {
           printf("%d ", *(p + t));
        }
        free(p);
     }
}
```

Bibliografia

- KERNIGHAN, Brian W. e RITCH, Dennis M., **C: A Linguagem de Programação**, Rio de Janeiro, Campus, 1986.

- PLAUGER, P.J. e BRODIE J. **Standart C : guia de referência básica**, São Paulo, Mcgraw-Hill, 1991. 306p.

- HANCOCK, Les e KRIEGER, Morris. **Manual de Linguagem C**, Rio de Janeiro, Campus,1985. 182p.

Bibliografia

- MIZRAHI, Viviane V. **Treinamento em Linguagem C** - módulo 1 e 2, São Paulo, McGraw-Hill, 1990, 241p.

- SCHILDT, Herbert. **Turbo C: Guia do Usuário**, São Paulo, McGraw-Hill, 1988, 414p.